

Concurrency & More Design Patterns

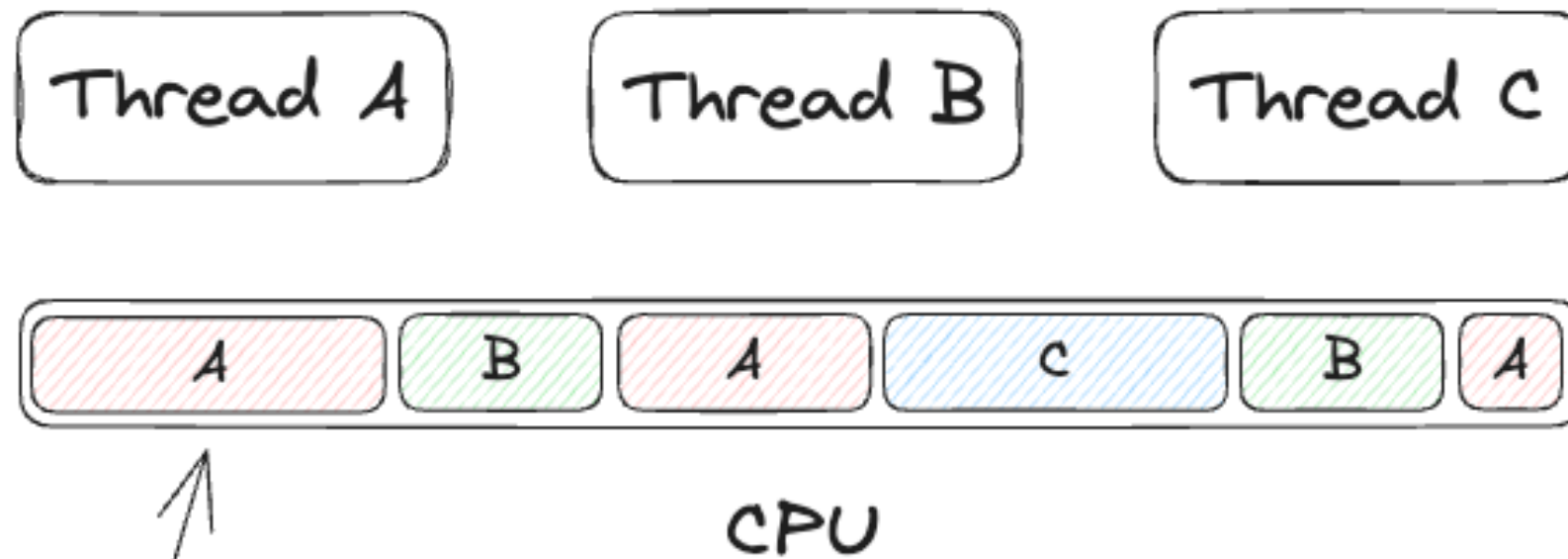
★ Software Design & Architecture ★



agenda



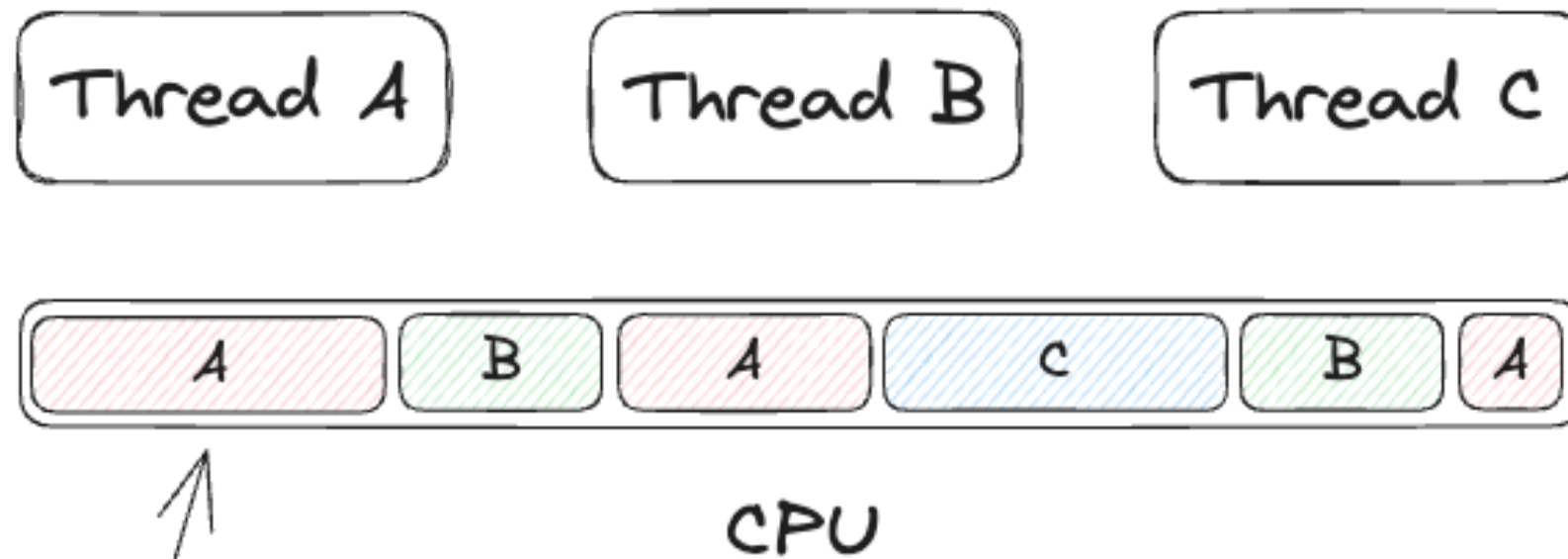
- Concurrency
- Data Races
- Singleton Pattern
- Abstract Factory Pattern
- Decorator Pattern



concurrency



- Concurrency: the ability of a program to execute different sections of code seemingly simultaneously.
- It is the basis behind all of modern computing, and it is how your computer is able to do so much all at once.



concurrency



- Threads: Sequences of instructions that the CPU can choose to execute at any given time (unpredictable)
- Different threads can be scheduled very closely to each other to give the illusion that they are running at the same time

★ single threading

- Up until now, most of the code you have written is single-threaded.
- Executes line after line, without interruption.

A **single chef** cooks a steak first, then the potatoes, then the veggies.

By the end, only the veggies are warm and we **wasted time** sitting around, as we waited for each prior dish to be finished before continuing.



multi- threading

- Different threads run different parts of your code.
- Creating multiple threads in your programs allows you to execute lines simultaneously, instead of one after the other.

Multiple chefs prepare different dishes at the same time. One chef does steak, another does potatoes, and another does the veggies.

All the food comes out warm and in a timely manner!



★ asynchronous

- Instead of spawning multiple threads to run code at the same time, we have **one thread that manages tasks**.
- The thread manages its tasks by having them **yield control when they are busy** doing something else. Once the task is ready to run again, we continue.

A **single chef** puts the steak on the grill.
While waiting, chef goes to boil the potatoes. While the potatoes boil, chef starts on the veggies. Chef moves between the tasks as they require attention.

All food comes out warm and in a timely manner!

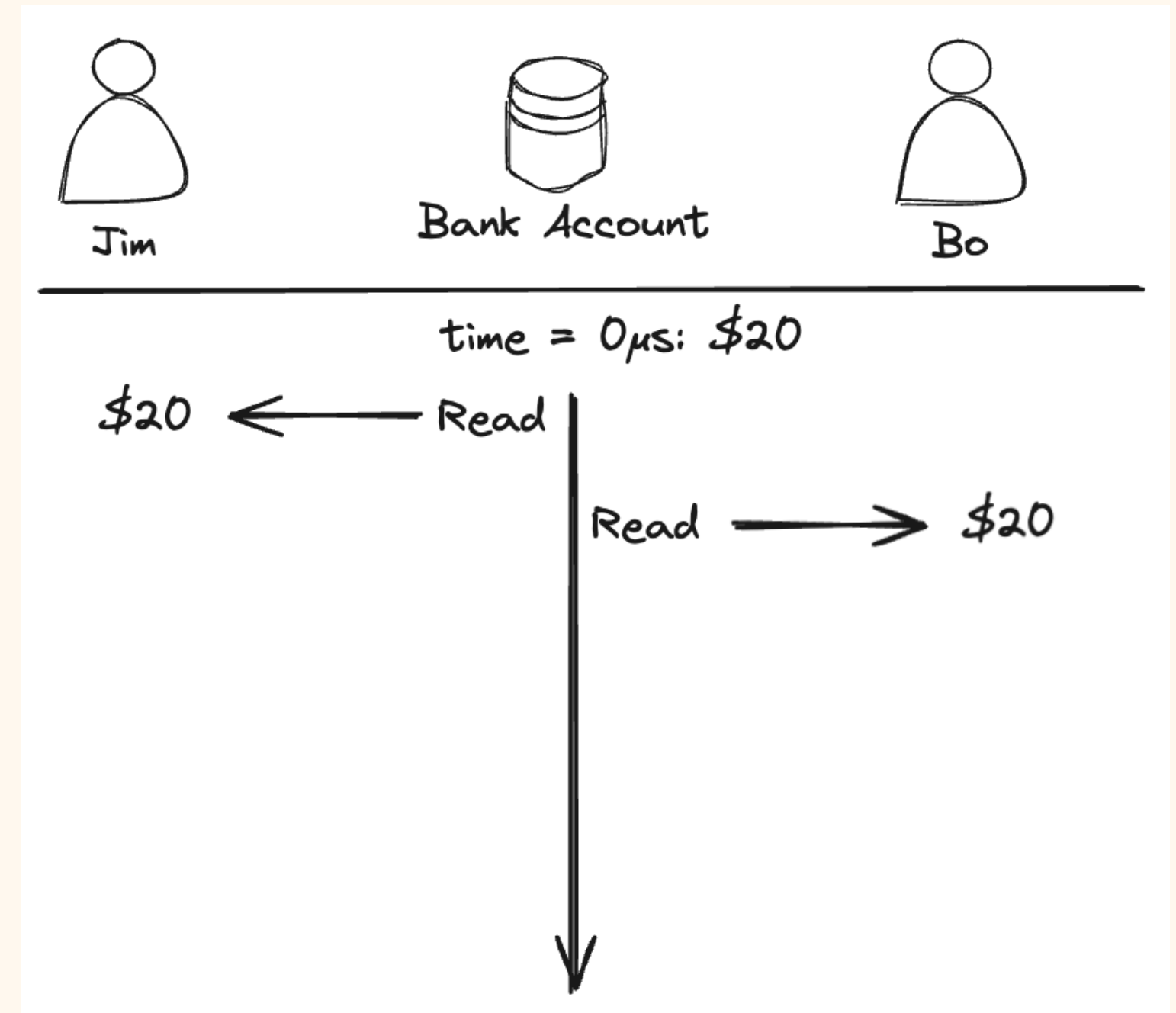


★ data races ★

Suppose Bo and Jim (threads) are accessing their shared bank account at the "same" time (transaction could be microseconds apart).

A person must read the current balance before depositing / withdrawing money from the account.

Jim and Bo both see \$20 in the bank account.

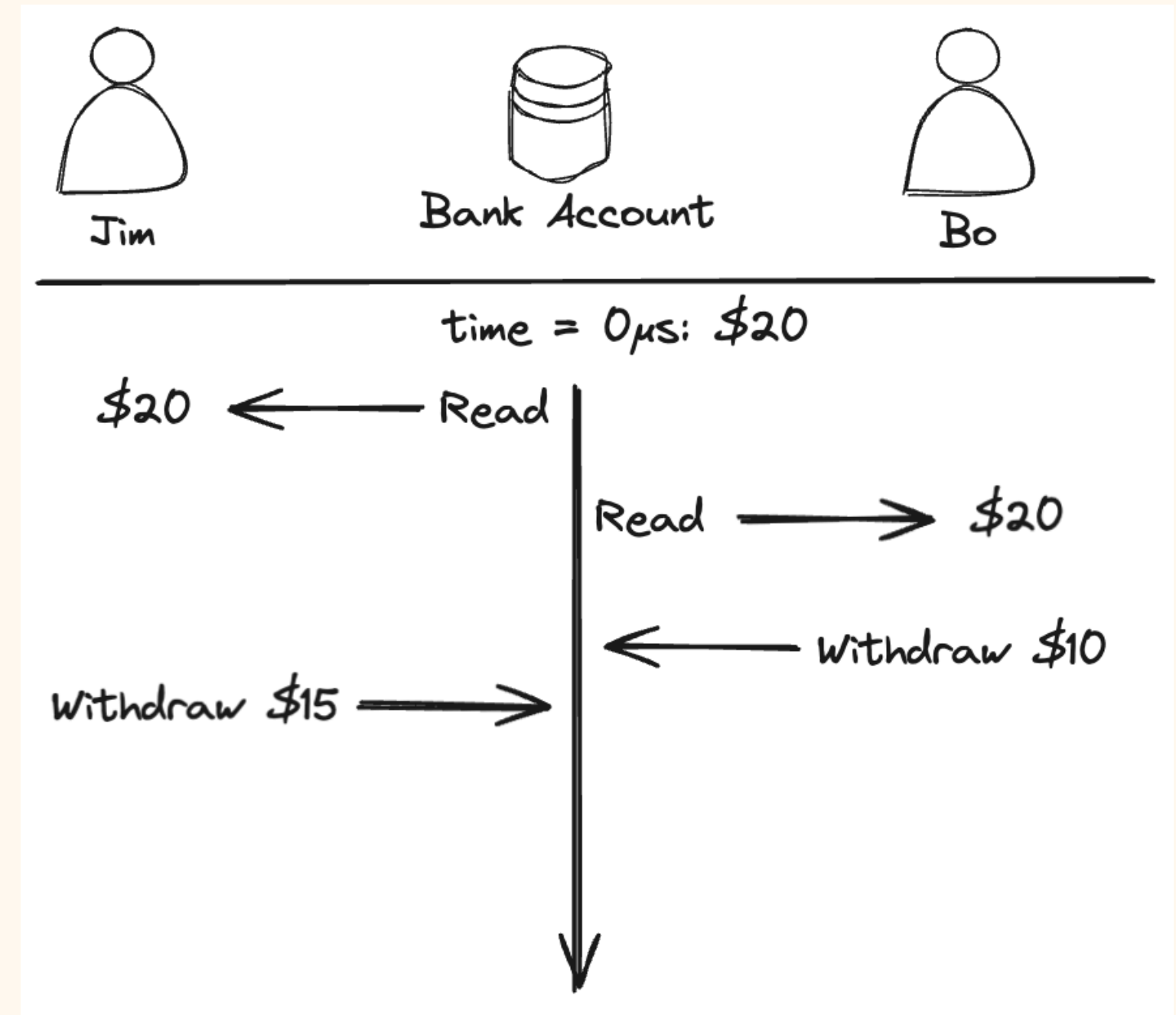


★ data races ★

1. Bo withdraws \$10 from the account.
2. But Jim also saw \$20, and so he rightly draws \$15 at the same time.

Uh oh...but we only had \$20 to begin with :(

This is a very common problem when implementing multithread code known as a **race condition**.

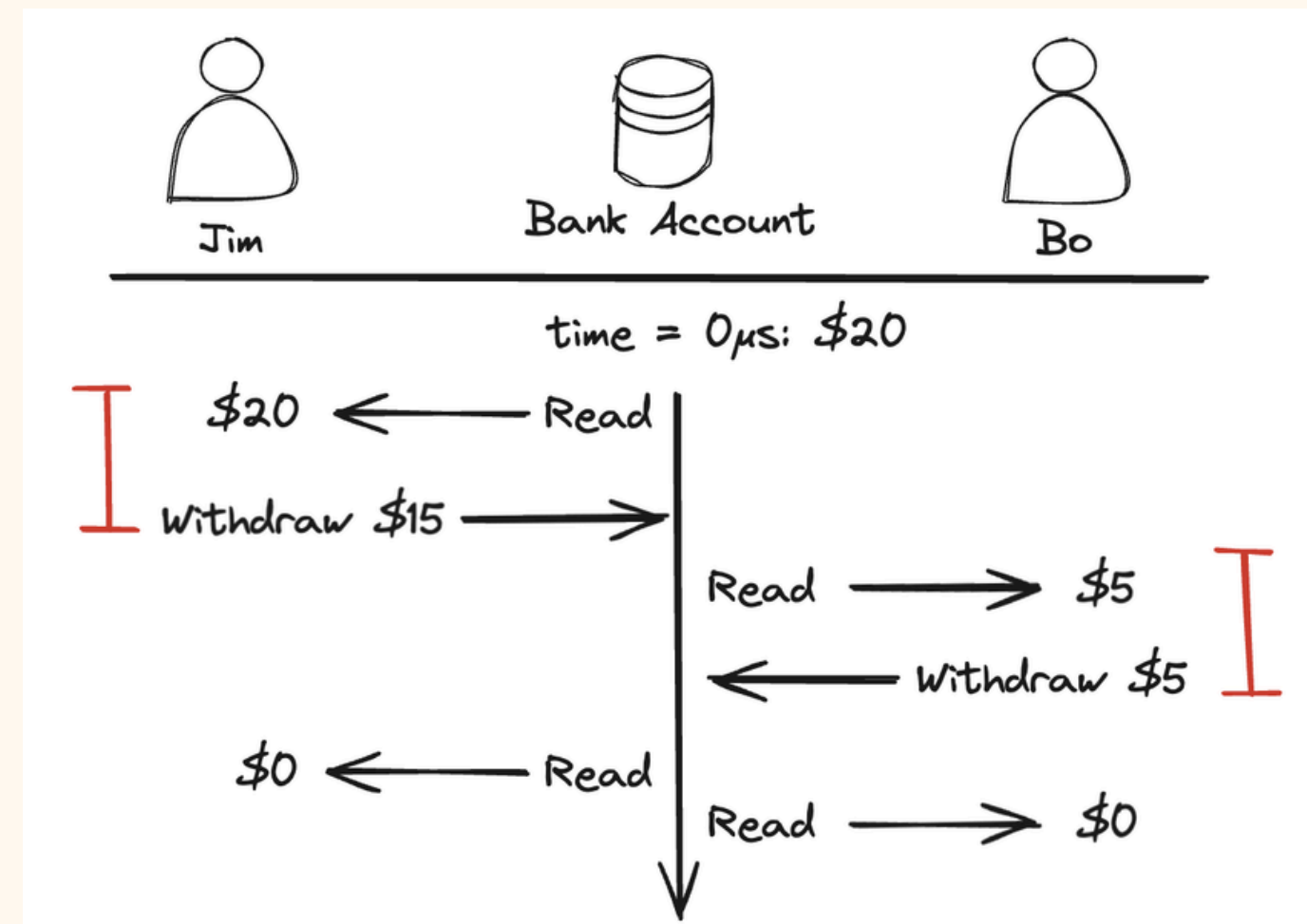


★ data races ★

- **Race condition** occurs when **two or more threads** attempt to read and write to a **shared value** at the **"same" time**.
- Outcome is **non-deterministic** and **dependent on the order** in which threads are scheduled on the CPU

Solution: Synchronise code that may read and write to a shared value.

- Enforce that a read + (withdraw or deposit) must be **completed before the next** can begin.
- **synchronized** keyword ensures that this region of code can only be touched by **one thread** at any single moment.





singleton pattern

Creational

the problem



Say we have a system where multiple parts create their own instance of a database connection. This leads to **inconsistent state**, duplicated data, and **resource waste/conflicts** (multiple database connections).

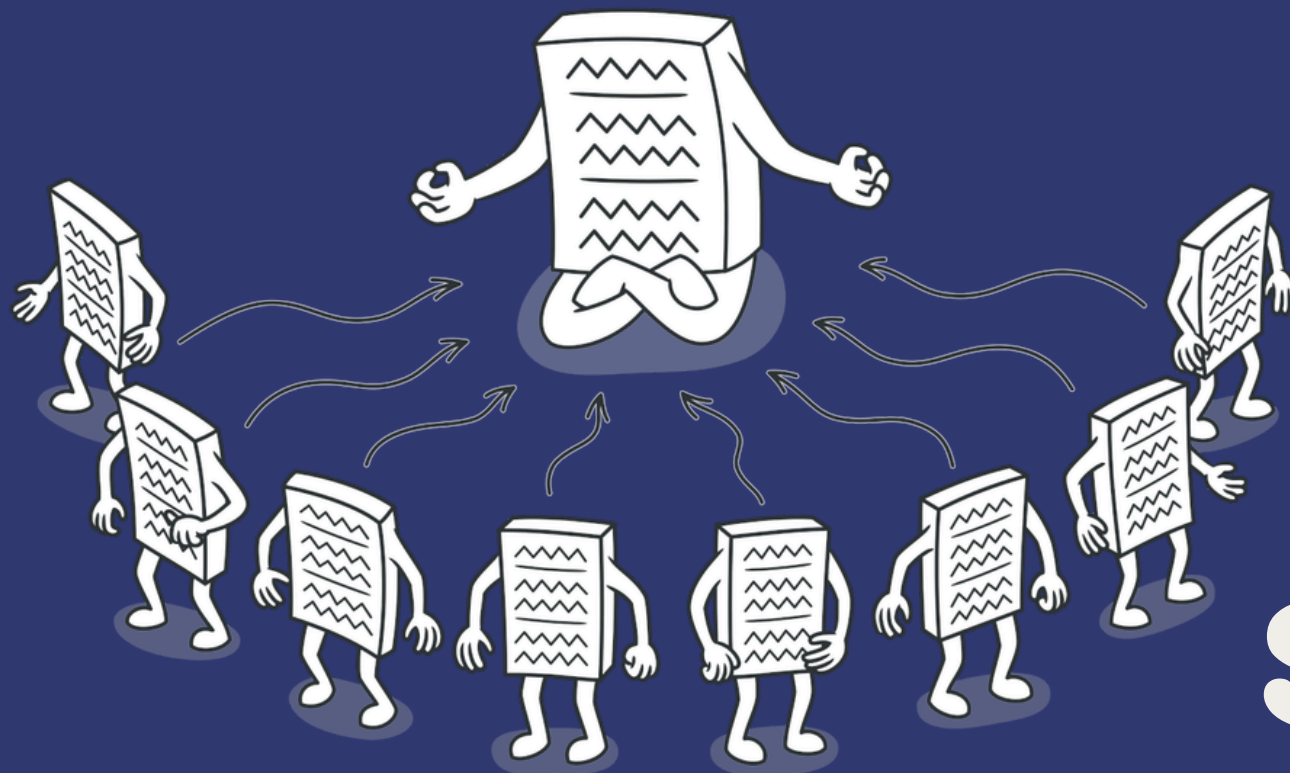
We need to ensure there is **only one instance** and a **single point of access**.



What is it?

Creational design pattern that specifies that:

- A class can only have one active instance
- Accessing the instance must be through a global access point



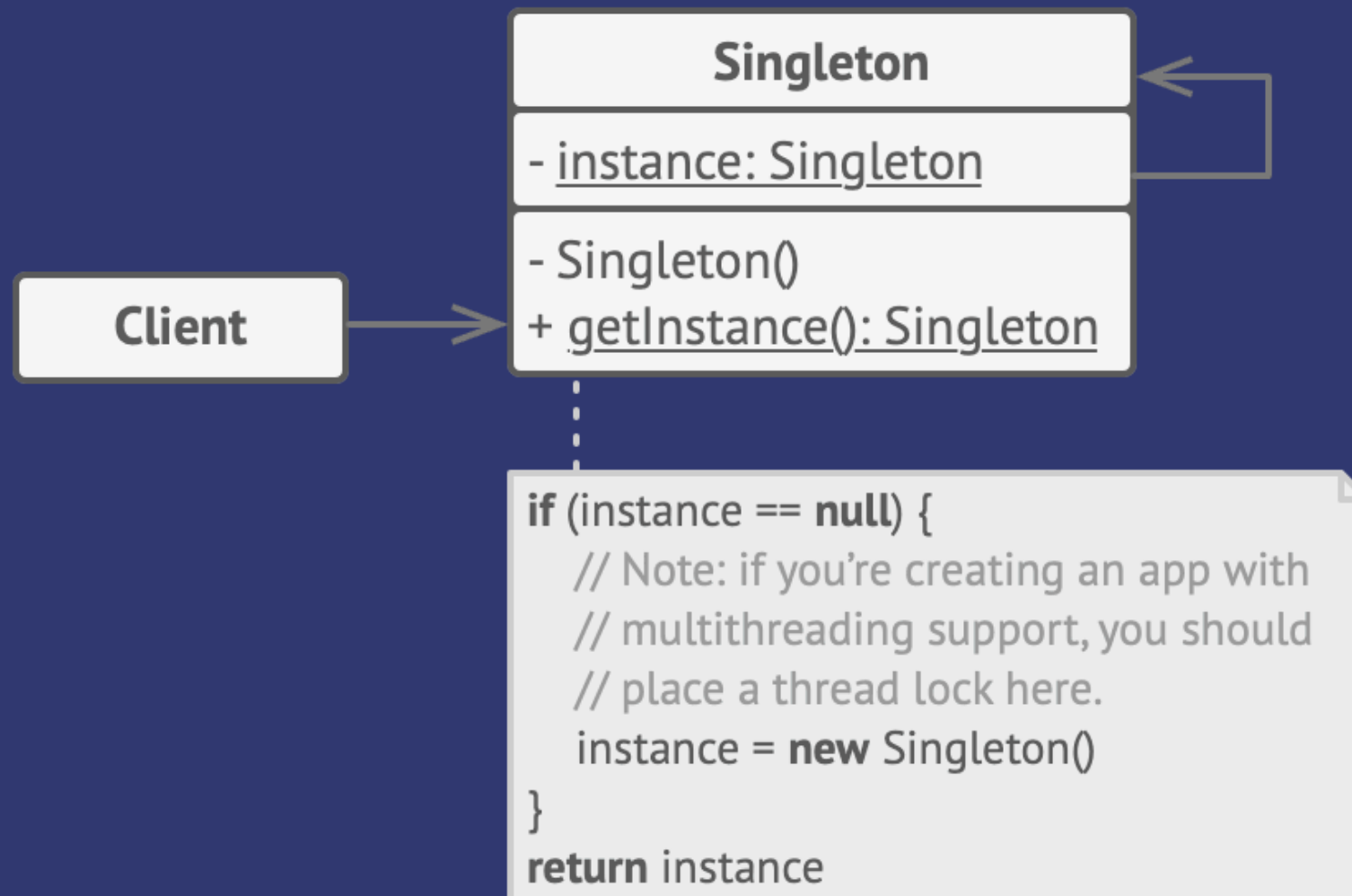
Why use it?

- Provides a **global access point** to a **single instance** of a class
- Prevents inconsistent state across multiple instances. Helps **manage shared resources** efficiently

When use it?

- Exactly 1 instance is required system-wide
- Multiple parts of the system need shared access to this same instance
- Creating multiple instances would cause errors or resource issues

singleton pattern



- The class holds a **static instance of itself via composition**
- All **constructors are private** to prevent arbitrary instances from being created
- Accessing the instance must be through the static method `getInstance`
- Class is **only instantiated once** when the `getInstance` is called for the first time.

singleton pattern



code demo!

Overcooked



overcooked

It's September 17th (the most common birthday in Australia), and all of our chefs are jostling to get birthday cakes out of the oven. In order to safely access the oven, each Chef needs to use the Oven Mitts. But there seems to be some strange behaviour where because each Chef has their own mitts, they grab cakes willy-nilly....

Currently, you will find that each **Chef accesses the oven at the same time** (which doesn't make sense). They also seem to just take out as many cakes as they can until the oven has none left, or somehow has a negative amount left??



abstract factory pattern

Creational

the problem



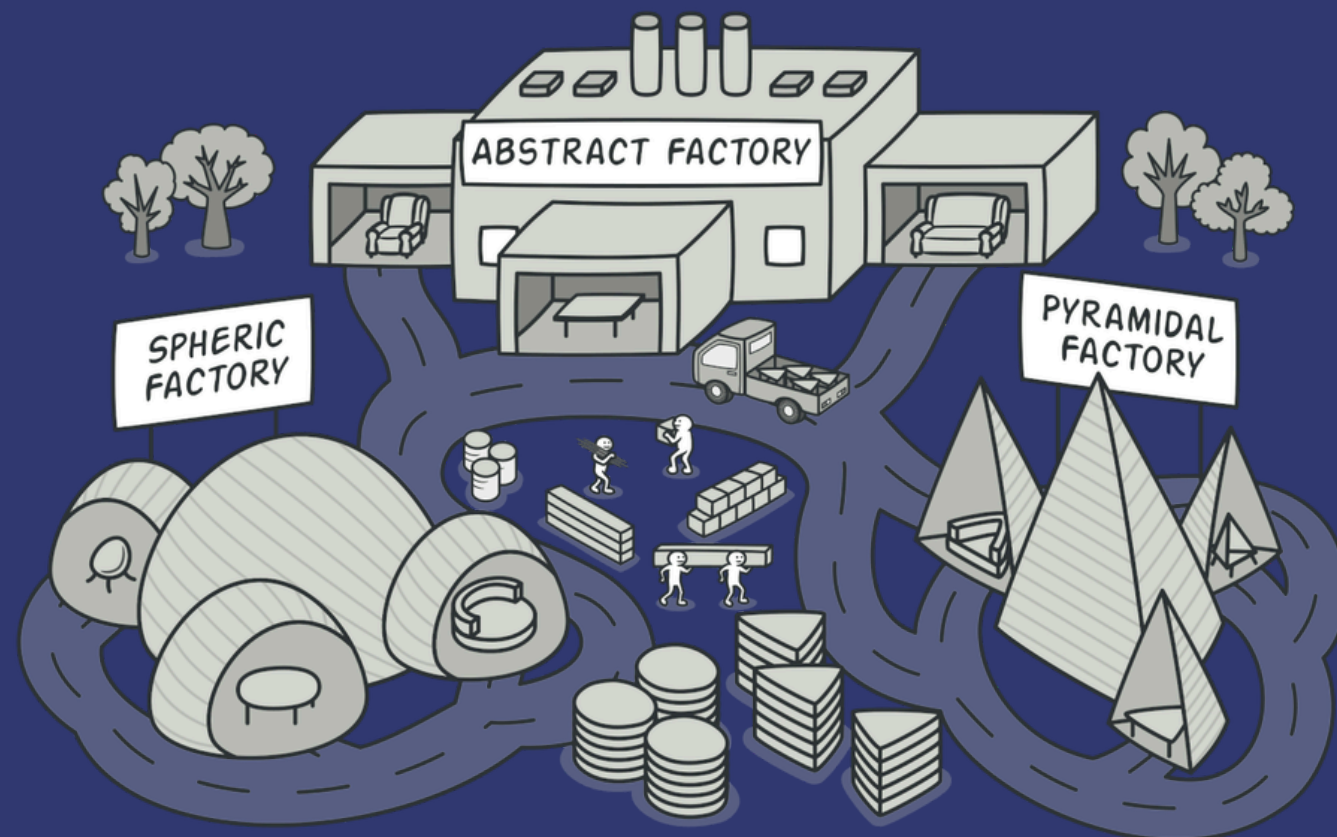
Suppose your app supports **different themes** (e.g. Dark, Light), and each theme needs **matching components** (Button, Menu, Window).

You risk **mixing incompatible components or scattering creation logic** everywhere. We need a way to create **consistent sets of related objects**.



What is it?

- Creational design pattern that provides an interface for creating families of related objects without specifying concrete classes
- Ensures **created objects are compatible** with each other



Why use it?

- Keeps object creation consistent across related types
- **Reduces coupling** to specific implementations

When use it?

- System needs to support multiple **related variants** (e.g. themes, platforms)
- You want to switch entire families of objects together
- Object creation logic is complex or grouped

abstract factory

thrones (again)

“Inside src/thrones, there is some code to model a simple chess-like game.

In this game different types of characters move around on a grid fighting each other. When one character moves into the square occupied by another they attack that character and inflict damage based on random chance.” (tut04)

thrones (again)

We are now changing the boardgame: now, **pieces can be made from different materials**: Wood, Plastic or Metal. We want to be able to pick and choose which material we create a board with. Each material has its own construction requirements:

- **Metal** pieces are heavy and hence can only be set with $0 \leq x \leq 5$, and $y = 0$
- **Plastic** pieces are lightweight and can be **placed anywhere within a given bound** (provided to the factory)
- **Wooden** pieces are large and bulky, and take up a $n \times n$ grid, meaning they can **only be placed on tiles with x and y values which divide n** (n is provided to the factory)

Utilise the Abstract Factory pattern to solve these requirements.



code demo!

Thrones





decorator pattern

Structural

the problem



Suppose you have a Coffee and want optional features (milk, sugar, syrup).

Using inheritance would require **many combinations** (e.g. MilkCoffee, SugarCoffee, MilkSugarCoffee)...this leads to **class explosion!**

We need a way to flexibly add features without creating many subclasses.





What is it?

- Structural design pattern that allows the **behaviour of objects to be enhanced** at runtime by placing them into special **wrapper objects**



Why use it?

- Enhance the behaviour of an object at **runtime** without violating OCP

When use it?

- Need to add behaviour to objects dynamically at runtime
- Many feature combos would cause too many subclasses (class explosion)
- Behaviour can be layered or combined flexibly (e.g. adding multiple features to an object)

decorator pattern



Where does the wrapped object go?

In the **constructor parameter**. It's stored as a **private/protected field** on the decorator instance.

Why can I pass a decorator into another decorator?

Because they all implement the same interface. The type system only cares that it's a Coffee (or whatever interface you're using), not what kind it actually is.

decorator FAQs

How does calling the method work?

The decorator calls the wrapped object's method first, gets the result, then adds its own behavior before returning.

Why composition and not inheritance?

Inheritance would require a new class to be made for every combination, since it's an 'is-a' relationship. Fixed, rigid, creates class explosion.

Composition = "I HAVE a Coffee inside me, and I add my own stuff". Flexible, reusable, scalable, original class doesn't change



1. Create a **component interface** and declare methods that are common to both the primary component and the optional layers.
2. Create a **concrete component class** and define the **base behaviour** in it.
3. Create a **base decorator class** with a field for storing a reference to a wrapped object (declared with component interface type). The base decorator must delegate all work to the wrapped object.
4. Make sure all classes implement the component interface.
5. Create concrete decorators by extending them from the base decorator. A concrete decorator must execute its behaviour before or after the call to the parent method (which always delegates to the wrapped object).
6. The client code must be responsible for creating decorators and composing them as needed.

decorator implementation



code demo!

Thrones



thrones (pt. 2)

- Suppose the game now necessitates support for **any character to wear different sorts of armour**.
 - **Helmet** reduces the amount of damage inflicted upon a character by 1 point.
 - **Chain mail** reduces the amount of damage by half (rounded down).
 - **Chest plate** caps the amount of damage to 7, but also slows the character down. If the character is otherwise capable of moving more than 3 squares (by manhattan distance) at a time, they cannot move.
- Assume that there are **no restrictions on the number** of pieces of armour a character can wear and that the "**order**" in which armour is worn affects how it works.



lab time!

SitarME306